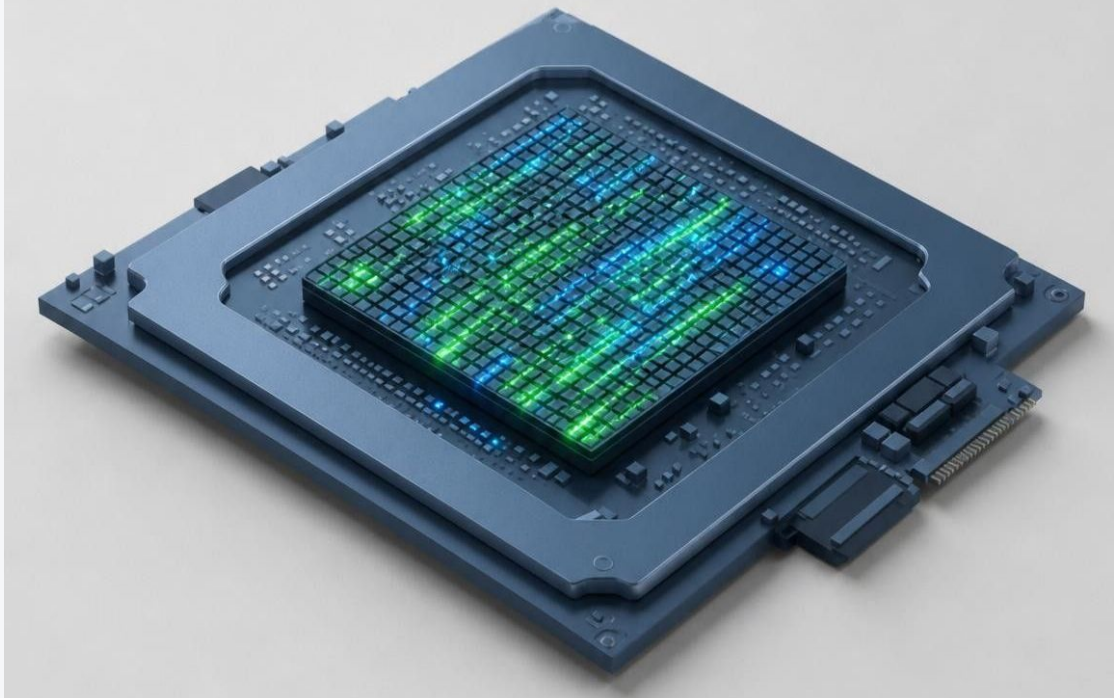
GPU kernels: the hidden programs that decide how fast AI runs

What kernels are, why they matter, and how NVIDIA, Triton, DeepSeek and Anthropic use them

We know the model must fit, and that LLMs spend most time moving bytes. Next question: who decides how those bytes move? **Kernels**, the small programs that run on the GPU. Two kernels can do the same math, give the same answer, and one can be **3× faster**.

KERNELS MEMORY SPEED



What we will cover

01

The basics

Where kernels fit in the series, what a kernel is, from model code to GPU work, thousands of cores, the GPU memory ladder

02

What makes a kernel fast

The golden rule, fusion, launch overhead and CUDA Graphs, tiling, tensor cores and precision

03

FlashAttention

The giant table problem, how FlashAttention avoids it, versions 1, 2 and 3

04

Who writes kernels

NVIDIA's libraries, why kernels are NVIDIA's biggest advantage, Triton, how PyTorch writes kernels for us

05

Case studies

DeepSeek, Claude on three chips, a fast kernel with wrong answers, Claude as a kernel engineer

06

Judging and choosing

How to judge a kernel, when to write one, cheat sheet

Where kernels fit in our GPU series

1	2	3	4
What is a GPU	**Will it fit**	**Why is it slow**	**Kernels**
Thousands of small cores at once	Weights + KV cache + overhead in VRAM	Moving bytes is the bottleneck	The programs that decide how bytes move and math is done

- Deck 3 assumed work was done as well as possible; the GPU only does what its kernels tell it
- A badly written kernel can read the same data from VRAM several times, write in-between results only to read them back, leave most cores idle, or use slow math units when faster ones exist

Deck 3's formulas give the **best possible** speed; kernels decide **how close we get**

That's why kernel engineers are among the most sought-after people in AI: a better kernel lowers the cost of every answer.

What is a kernel?

A small program that runs on the GPU, where thousands of cores run the same program at once, each on its own small piece of data.

- Job: add 1 to each of 1 million numbers. A CPU core walks the list one by one; thousands of GPU cores run "read my number → add 1 → write it back" at once
- A kernel usually does one job: a matrix multiplication, a softmax, adding a bias
- An LLM is many kernels run one after another for every layer; one token in a 32-layer model may run hundreds
- Not the operating system kernel: in GPU programming it just means "the function that runs on the GPU"

"The model runs on the GPU" means a long chain of kernels runs on the GPU.



From one line of model code to work on the GPU

1. We write "multiply table A by table B"
2. PyTorch checks table kind, size, precision, and GPU
3. It picks a ready-made kernel for this exact case, often from NVIDIA's cuBLAS
4. The CPU sends a launch request: which kernel, which data, how many copies
5. The GPU starts the kernel on thousands of cores
6. Results are written to VRAM; PyTorch launches the next kernel

The CPU is the manager: it picks the next kernel and tells the GPU to start it

Kernel choice matters: a library may hold dozens of kernels for one operation, each tuned for a size, precision or GPU

Every kernel reads from and writes back to VRAM: the root of the biggest waste in this deck

How a kernel runs on thousands of cores

Thread

One running copy of the kernel, handling a number or a few

Block

A group of threads, often a few hundred, sharing a small, very fast memory

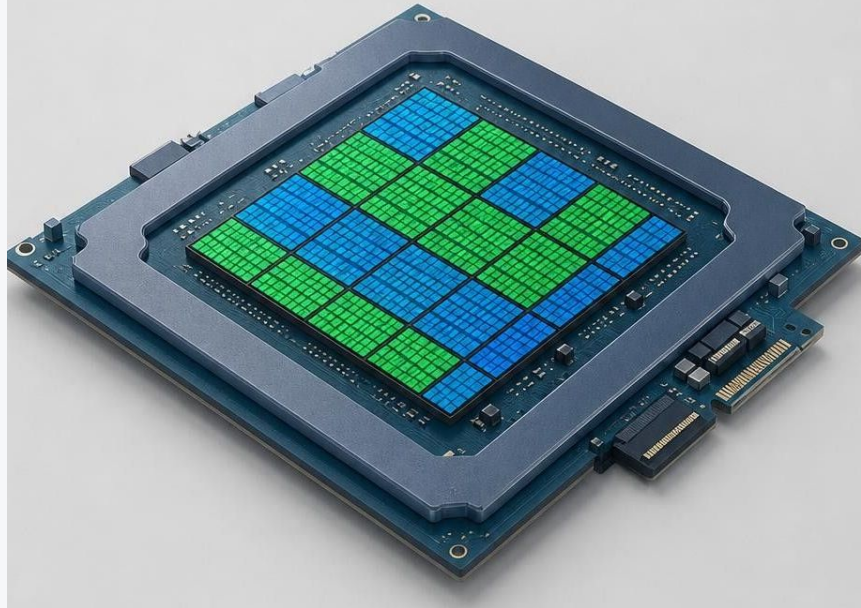
SM

Streaming Multiprocessor: a chip section that runs blocks. H100 has **132 SMs**, each with cores, fast memory and tensor cores

1	2	3	4	5
Whole job	Split into blocks	To the 132 SMs	Threads run together	Finished SM takes next block

Why it matters: only 50 blocks leaves 82 of 132 SMs idle; uneven blocks make SMs wait; threads waiting on VRAM stall the SM.

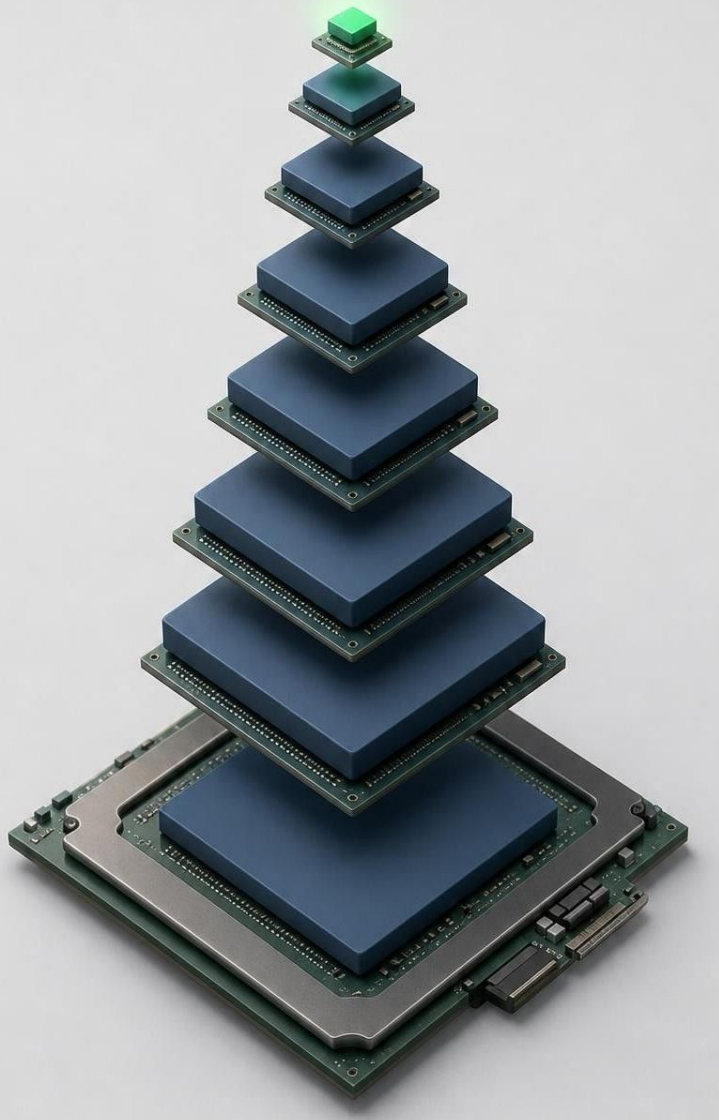
A good kernel keeps every SM busy all the time.



The kernels inside one layer of an LLM

Step	What happens	Kind of kernel	Usually limited by
1	Normalise the input (RMSNorm)	Step-by-step	Memory
2	Make the Query, Key and Value	Matrix multiplication	Memory in decode
3	Add position info (rotary embedding)	Step-by-step	Memory
4	Attention: compare earlier tokens, mix Values	Attention kernel	Memory in decode
5	Project the attention output	Matrix multiplication	Memory in decode
6	Add back the input (residual)	Step-by-step	Memory
7	Normalise again (RMSNorm)	Step-by-step	Memory
8	Expand (feed-forward up and gate)	Matrix multiplication	Memory in decode
9	Apply the activation function	Step-by-step	Memory
10	Shrink back (feed-forward down)	Matrix multiplication	Memory in decode
11	Add back the input again (residual)	Step-by-step	Memory

- Two families: big matrix multiplications (2, 5, 8, 10) hold the weights and move most bytes; small step-by-step kernels do little math but still read and write VRAM
- In decode nearly everything is memory-bound; 32 layers = **over 350 kernels per token**
- Two opportunities: make big matrix kernels move bytes efficiently, and stop small kernels wasting VRAM trips



The GPU memory ladder: tiny and fast to big and slow

Memory level	Where it is	Size on H100	Speed
Registers	Inside each core	~256 KB per SM	Fastest of all
Shared memory (SRAM)	Inside each SM	Up to ~228 KB per SM	Extremely fast
L2 cache	On the chip, shared	~50 MB	Fast
VRAM (HBM)	Next to the chip	80 GB	~3.35 TB/s

- All fast on-chip memory together is only tens of MB; VRAM is 80 GB, thousands of times bigger
- Weights live in VRAM; math happens on data in registers
- Data must climb: VRAM → L2 → shared memory → registers → math; every climb costs time

ⓘ Fast memory at the top is where data should stay as long as possible, reused without another VRAM trip.

The golden rule of fast kernels

Bring data up from VRAM as few times as possible, and use it as many times as possible while it is close to the cores.

H100 HANDLES

~295

math steps per byte; fewer means memory-bound

Recall: arithmetic intensity = math steps ÷ bytes moved.

Way 1, move fewer bytes

Don't write in-between results to VRAM, use smaller number formats

Way 2, more math per byte

Load a piece of a table into shared memory once, use it for many calculations

Bad: load → use once → load again → use once. **Good:** load into shared memory → use 100 times → done

The tricks that follow the rule: **fusion, fewer launches, tiling, special math units**.

The waste of separate kernels

After a matrix multiplication, data (say 1 GB) often goes through three small steps: **scale**, **add a bias**, apply an **activation** like GELU.

1

Kernel 1 · scale

Read 1 GB → write 1 GB

2

Kernel 2 · add bias

Read 1 GB → write 1 GB

3

Kernel 3 · activation

Read 1 GB → write 1 GB

TOTAL MOVED

6 GB

H100: 6 GB ÷ 3,350 GB/s ≈ 1.8 ms

- After each kernel, the next reads the same numbers straight back: a pointless round trip
- Each step does 1–2 operations per number, so all three are completely memory-bound

⚠ This pattern appears dozens of times in every layer of every model.

Kernel fusion: three steps in one trip

Fusion combines several small kernels into one: read once, do every step in fast memory, write only the final result.

Fused: read 1 GB → scale → add bias → activation (all in registers) → write 1 GB. Total moved: 2 GB.

Approach	Data moved	Time on H100
Three kernels	6 GB	~1.8 ms
One fused kernel	2 GB	~0.6 ms

- Used in LLMs for: normalisation with nearby steps, activation with the preceding matrix multiplication, next-token selection (temperature, softmax, top tokens), and attention itself (FlashAttention)
- The first thing most AI compilers try automatically

SAME MATH, SAME ANSWER

3×

faster, only from moving a third of the bytes

Kernel launch overhead and CUDA Graphs

- Each kernel launch has a small fixed delay of a few **microseconds**: the CPU prepares and sends it, the GPU sets it up
- 32 layers × 10+ kernels ≈ **300–400 kernels per token**; at ~5 µs each ≈ **1.5–2 ms** just starting kernels
- Llama 3 8B on an H100 spends ~4.8 ms moving weights per token, so launches can add about a third; for small models, launch delay can exceed the real work

WITHOUT

CPU launch → kernel → CPU launch → kernel…

WITH CUDA GRAPHS

One replay → kernel kernel kernel…

CUDA Graphs: record the whole kernel sequence once, then the CPU sends one request, "replay this chain", and the GPU runs kernels back to back.

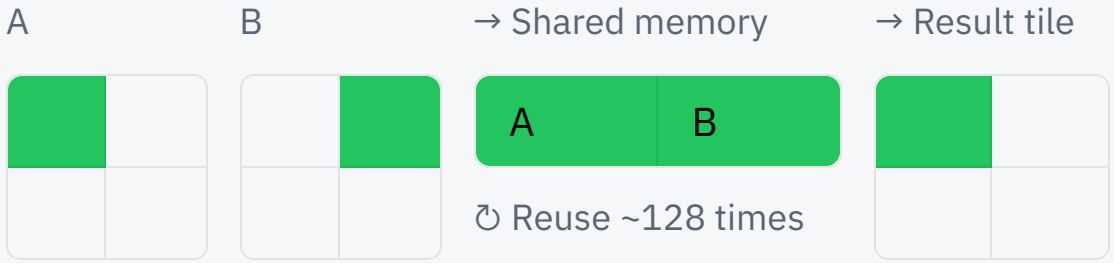
vLLM and TensorRT-LLM use CUDA Graphs during decode, removing most launch delay per token.

Tiling: how matrix multiplication kernels reuse data

Matrix multiplication is AI's most important kernel; each number in A is needed once per column of B, each in B once per row of A. **Naive kernel:** each thread fetches its row and column from VRAM, so the same numbers are fetched over and over.

Tiled kernel

1. Split A and B into tiles (e.g. 128 × 128)
2. A block loads one tile of each into shared memory once
3. All threads reuse them for many multiplications
4. Load the next pair
5. Write the result tile once



Arithmetic intensity rises ~128×: each loaded number is used ~128 times instead of once.

That's why large matrix multiplications reach **80–90%** of peak math speed while step-by-step ops stay memory-bound.

Tricky part: too-small tiles reuse too little; too-big ones don't fit in shared memory. The best size depends on GPU, table sizes and precision, so libraries keep many versions.

Tensor cores and precision-specific kernels

CUDA cores

One multiply-and-add at a time

Tensor cores

A whole small matrix multiplication, like 16 × 16, in one step

- The H100's ~990 TFLOPS comes from its tensor cores
- Kernels must be written to feed them: exact data layout, fast enough arrival, and each generation's own instructions

GPU generation	Year	New tensor core format
Pascal / Volta	2016–2017	FP16
Ampere (A100)	2020	BF16
Hopper (H100)	2022	FP8
Blackwell	2024	FP4

Each halving of number size can roughly double math speed, if the kernel uses it. So a new GPU's full speed isn't there on day one: until new kernels are written, it runs old ones and uses only part of its power.

The giant table problem inside attention

- Attention scores every pair of tokens. At **8,192 tokens**: 8,192 × 8,192 ≈ 67 million scores ≈ **134 MB** in FP16, per head, per layer
- Standard attention as separate kernels: Kernel 1 computes all scores → writes the table to VRAM; Kernel 2 reads it → softmax → writes it back; Kernel 3 reads it → multiplies by Values → writes output
- The same fusion waste, on a huge scale, every layer
- Worse with length: twice the text = **4× the table**, since it grows with the square; 32,000 tokens is 16× bigger than 8,000

⚠ For years this table was the main reason long context was slow and memory-hungry.

32 HEADS × 134 MB

4.3 GB

of scores per layer at 8,192 tokens

How FlashAttention avoids the giant table

2022, Tri Dao and colleagues at Stanford: **never write the full score table to VRAM. Compute it in small tiles, use each right away, throw it away.**

1	2	3	4	5	6
Small group of Queries	Load a tile of Keys and Values into shared memory	Score this tile	Softmax, multiply by Values in fast memory	Add to running output, next tile	Write only the final output

The softmax problem

It needs the row's largest score and total, but a tile shows only part of the row

The fix

Keep a running maximum and total; when a tile has a bigger maximum, rescale earlier partial results; the end result is exactly the same

Gives

No giant table in VRAM, attention memory grows in a straight line with length, far fewer bytes moved

Very important: not an approximation. Same math, same answer; only the order of work and data movement changed.

Case study 1: FlashAttention 1, 2 and 3

FlashAttention (2022)

Attention fused into one kernel that never writes the score table; noticeably faster training for GPT-2 and BERT in the paper's tests; much longer context practical

FlashAttention-2 (2023)

Split work across SMs better, keeping more of the chip busy; became the open-source standard

FlashAttention-3 (2024)

Built for H100 features. FA-2 reached only ~**35%** of H100 math power vs **80–90%** for tuned matrix kernels. FA-3 overlaps data movement with math, lets thread groups specialise (some fetch, others calculate), adds FP8

FA-3 ON H100

75%

of peak FP16 math, ~1.5–2× faster than FA-2

Same GPU, same model, same math, same answer → only the kernel changed → **several times faster, much less memory**. Every major LLM serving tool uses FlashAttention or a kernel built on its ideas.

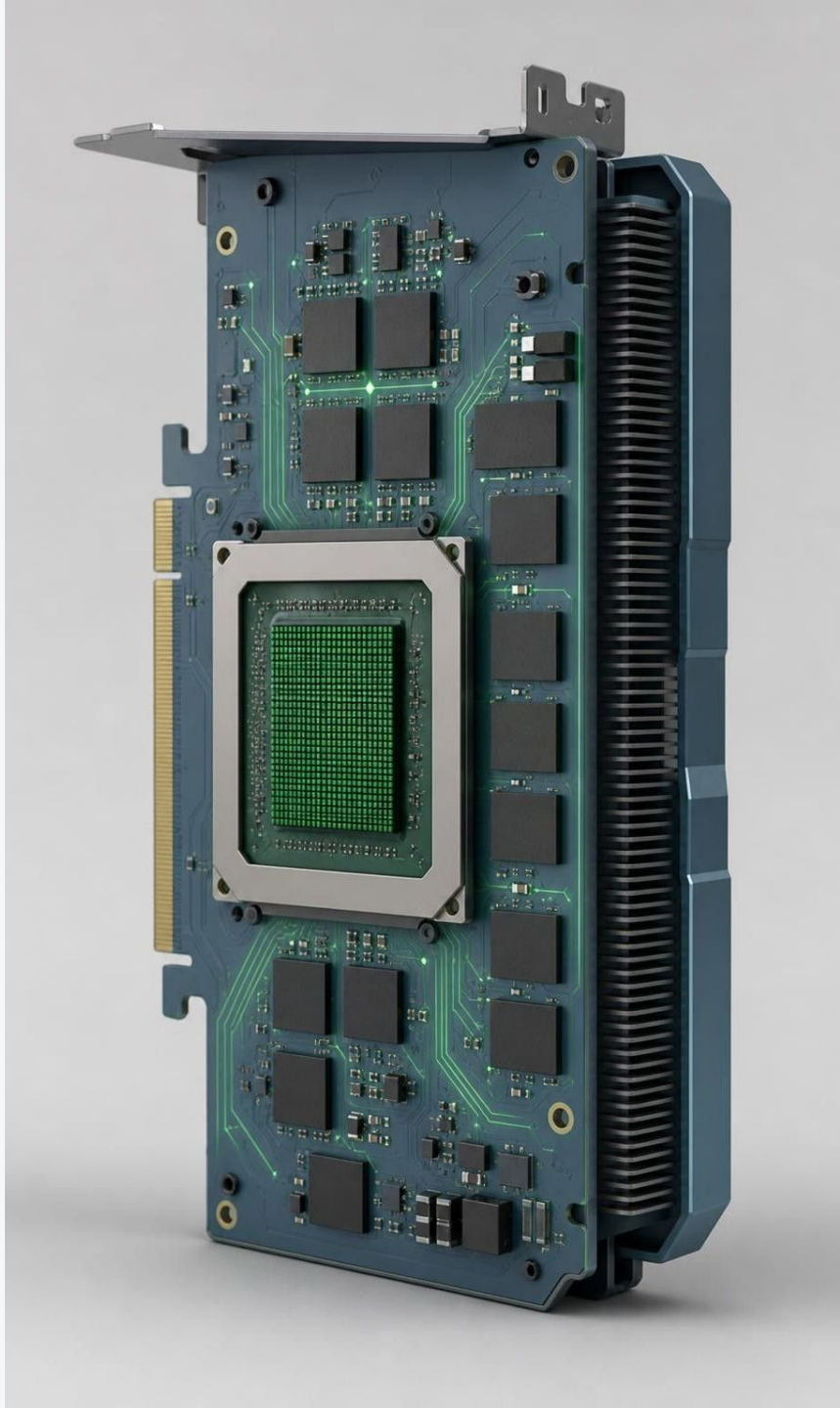
How NVIDIA uses kernels: the CUDA platform

Much of NVIDIA's strength is software, and most of that software is kernels. **CUDA** (first released 2007) is the language and tools for writing NVIDIA kernels; almost every AI framework, PyTorch included, sits on it.

Library	What it provides
cuBLAS	Matrix multiplication kernels, the most used in AI
cuDNN	Neural network layers: convolution, normalisation, attention
CUTLASS	Building blocks for writing your own fast matrix kernels
NCCL	Kernels for GPUs to send data to each other across many GPUs

- A PyTorch matrix multiply usually calls cuBLAS, which holds many versions tuned per size, precision and generation
- Each new GPU ships with updated libraries using its new tensor cores and formats

Why NVIDIA invests: a GPU is only as fast as its kernels; excellent built-in kernels give every PyTorch user speed for free.



NVIDIA case study: TensorRT-LLM

NVIDIA's open-source tool for serving LLMs as fast as possible on its GPUs, nearly every idea in this deck working together.

Fusion: many small transformer steps combined into single kernels

Fastest kernel per case: tries many candidates for each matrix multiplication on the actual GPU, keeps the fastest

Low-precision kernels: FP8, INT4 and newer formats

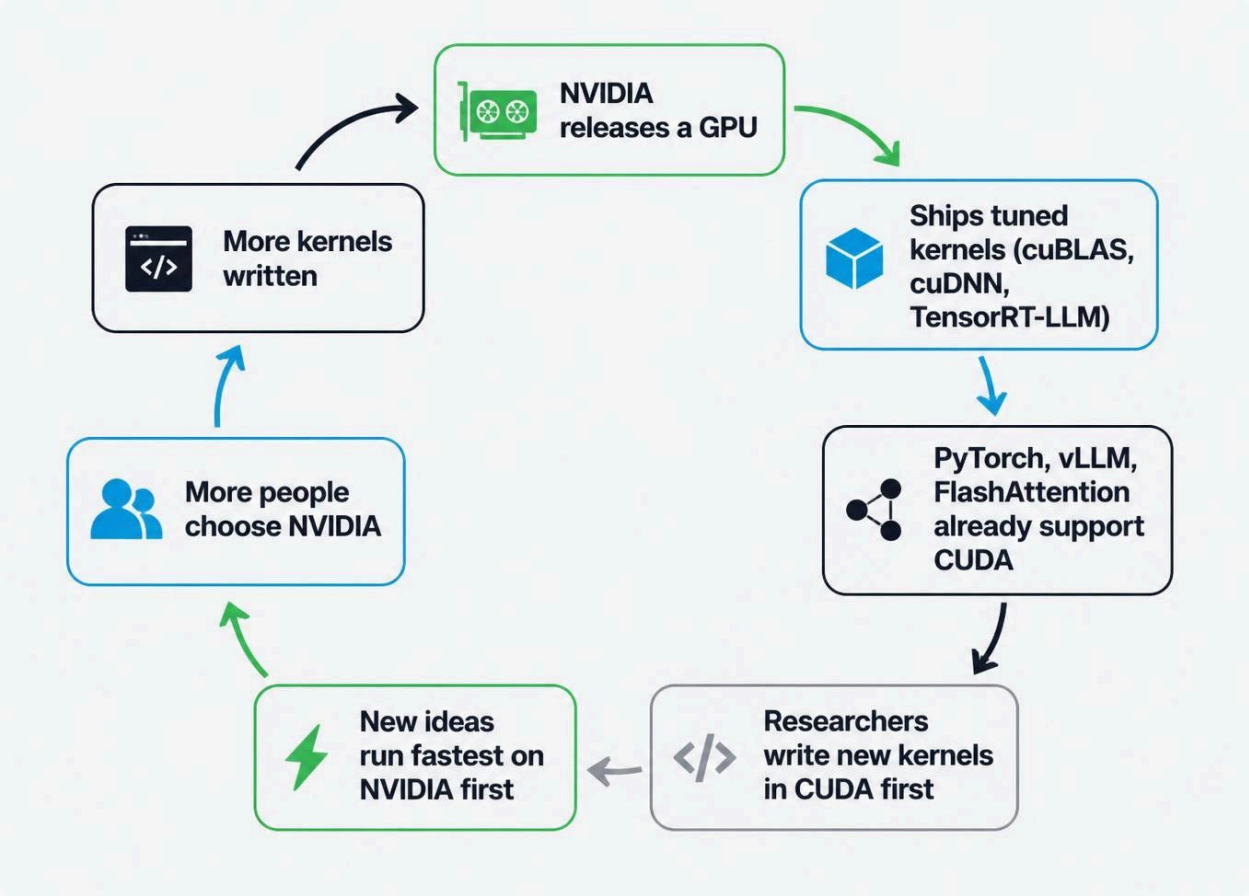
Custom attention: its own FlashAttention-style kernels

Serving features: batching and a paged KV cache

Flow: trained model → analysed for one GPU type and precision → fuse steps, pick fastest kernels → an engine tuned for that GPU → serves users

Trade-off: more setup and expertise, tuned to one setup, but closer to the GPU's real limits. NVIDIA also releases its best inference kernels through **FlashInfer**, usable by vLLM and SGLang.

Why kernels are NVIDIA's biggest advantage



- AMD, Google (TPUs), Amazon (Trainium) and startups make AI chips, some impressive on paper
- NVIDIA's edge is its kernel ecosystem, a cycle running since around 2007
- **For other chip makers:** good hardware isn't enough; they need kernel libraries, compilers, speed-measuring tools, and engineers willing to learn their platform
- **For us:** listed chip speed is half the story; the other half is whether excellent kernels exist for our model on that chip today

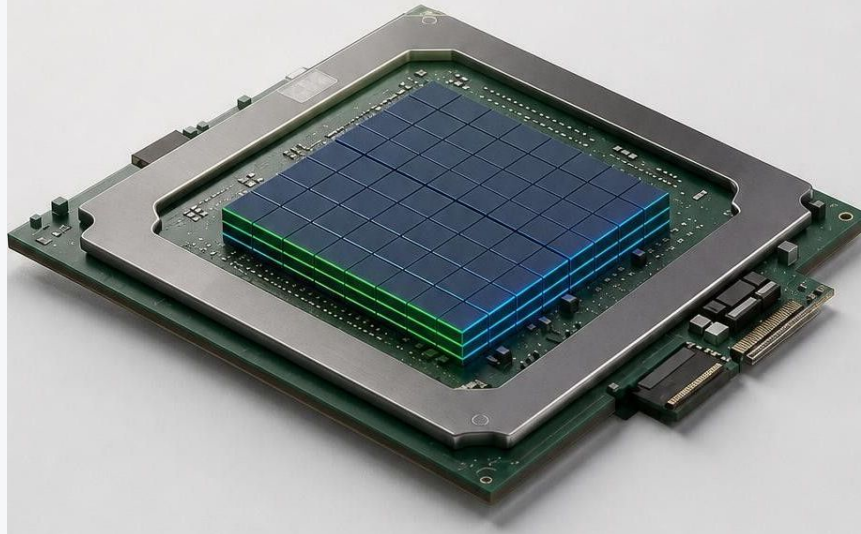
That's why tools that make kernels easier to write, like Triton, matter so much.

What is Triton?

- Writing fast CUDA kernels is hard: individual threads, hand-managed shared memory, avoiding memory conflicts, re-tuning per GPU. Few do it well
- **Triton:** open-source language and compiler released by **OpenAI in July 2021** to make fast kernels much easier; started as Philippe Tillet's PhD research at Harvard, continued at OpenAI
- **OpenAI's launch claims:** researchers without CUDA experience often got close to expert code; an FP16 matrix multiply in Triton could match cuBLAS in a short program; Python-like style
- Started on NVIDIA, now also supports AMD: one kernel fast on more than one chip

Core idea: think in blocks, not threads

CUDA: "thread 37, load this number, store it in shared memory, wait, then multiply..." Triton: "multiply this block by that block, add the result here." The compiler handles thread sharing, data movement and conflicts.



Triton vs CUDA: what changes and what doesn’t

Point	CUDA	Triton
Who it's for	GPU specialists	ML engineers and researchers
Engineer thinks about	Each individual thread	Blocks of data
Moving data to shared memory	Done by hand	Mostly handled by the compiler
Language style	C++	Python-like
Peak speed possible	Highest, with expert effort	Often close to CUDA for common kernels
Fine hardware control	Full	Less, by design
Runs on	NVIDIA only	NVIDIA and AMD

Unchanged

The golden rule. You still choose what to load once and reuse, what to fuse, block sizes, and whether it's memory- or compute-bound

CUDA still wins

For the hottest kernels needing every last percent: FlashAttention-3 was built on CUTLASS to reach H100 features

Today's pattern

Triton for most custom kernels, CUDA and CUTLASS for the few needing absolute peak speed

How PyTorch now writes kernels for us

With **torch.compile**, PyTorch writes new kernels automatically, in Triton.

Flow: Compile the model → PyTorch records the chain of operations → its compiler, TorchInductor, finds steps to fuse → writes a Triton kernel per fused group → Triton makes GPU instructions → big matrix multiplies may still call cuBLAS if faster → the model runs on the new fused kernels.

Why it's big

Fused kernels used to be hand-written; now the compiler finds and writes them, often from a one-line change

Good at

Fusing step-by-step ops (like scale, bias, activation), fusing normalisation with nearby steps, trying versions and keeping the fastest

Not good at yet

Inventing algorithms like FlashAttention's running softmax, which still needs humans; compiling takes time and the speed-up is sometimes small

Many of the most important kernels in open-source LLM serving are written in Triton, by humans or compilers.



Case study 2: DeepSeek and the power of kernels

DeepSeek trained and served V3 and R1 on NVIDIA **H800**s: an H100 with slower GPU-to-GPU links due to US export rules, and far fewer GPUs than top US labs. It won by writing its own kernels, open-sourced in its February 2025 "Open-Source Week".

- **FlashMLA:** decode attention kernel for DeepSeek's MLA (Multi-head Latent Attention); ~3,000 GB/s on an H800 with ~3,350 GB/s
- **DeepGEMM:** FP8 matrix multiplication; over **1,350 TFLOPS** on Hopper; core logic ~**300 lines**, beating expert-tuned kernels for most sizes in DeepSeek's tests
- **DeepEP:** kernels sending tokens between GPUs to their chosen experts in MoE models, attacking the H800's slow links

90%

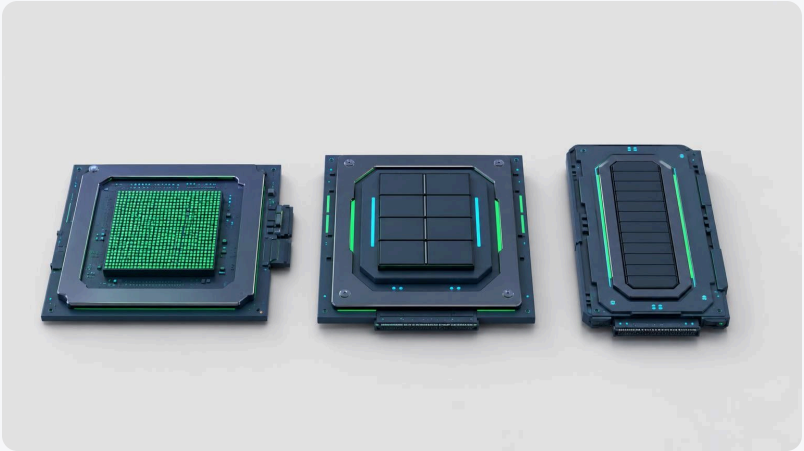
of maximum memory speed, FlashMLA

Lessons: kernels can partly make up for weaker hardware; kernels for your own model design beat general ones; some labs treat kernel engineering as core strategy.

How Claude uses kernels: one model, three kinds of chips

- Anthropic runs Claude on **NVIDIA GPUs**, **Google TPUs** and **AWS Trainium**; each needs its own optimisations, with strict standards that the model behaves the same on all
- A kernel is written for one chip: a CUDA kernel for an H100 can't simply run on a TPU or Trainium, each with its own memory ladder, math units and tools

Platform	Main way kernels are written or generated
NVIDIA GPUs	CUDA, CUTLASS, Triton, NVIDIA libraries
Google TPUs	The XLA compiler, turning the model into TPU instructions
AWS Trainium	AWS Neuron tools, incl. a low-level kernel interface (NKI)



Scale: AWS's Project Rainier, active late 2025, is nearly **500,000 Trainium2 chips** for training Claude; at AWS re:Invent 2025, Anthropic's Trainium inference lead presented its kernel work.

Lesson: at this scale kernel work is done per chip family, kept in step, and redone each new generation.

Case study 3: when a fast kernel gave wrong answers

Background

To pick the next token, the model finds the **top-k** most likely tokens and samples from them. Exact top-k is always correct but was slower; approximate top-k is much faster but may be slightly imprecise

What went wrong

On TPUs Claude used approximate top-k for speed. In August 2025 a rewrite of token-selection code, partly to fix precision issues, triggered a hidden bug in the XLA:TPU compiler. It sometimes returned completely wrong results, only for certain batch sizes and settings; confirmed for Claude Haiku 3.5, possibly others

Hard to find

Only in some situations, correct on a CPU, and the model often recovered from one bad token

Fix

Switched to exact top-k, standardised more operations on FP32, accepted a small speed loss (quality first), worked with Google's XLA team on the compiler

A faster kernel is only better if it's right.

Test every optimisation for correctness, not just speed.

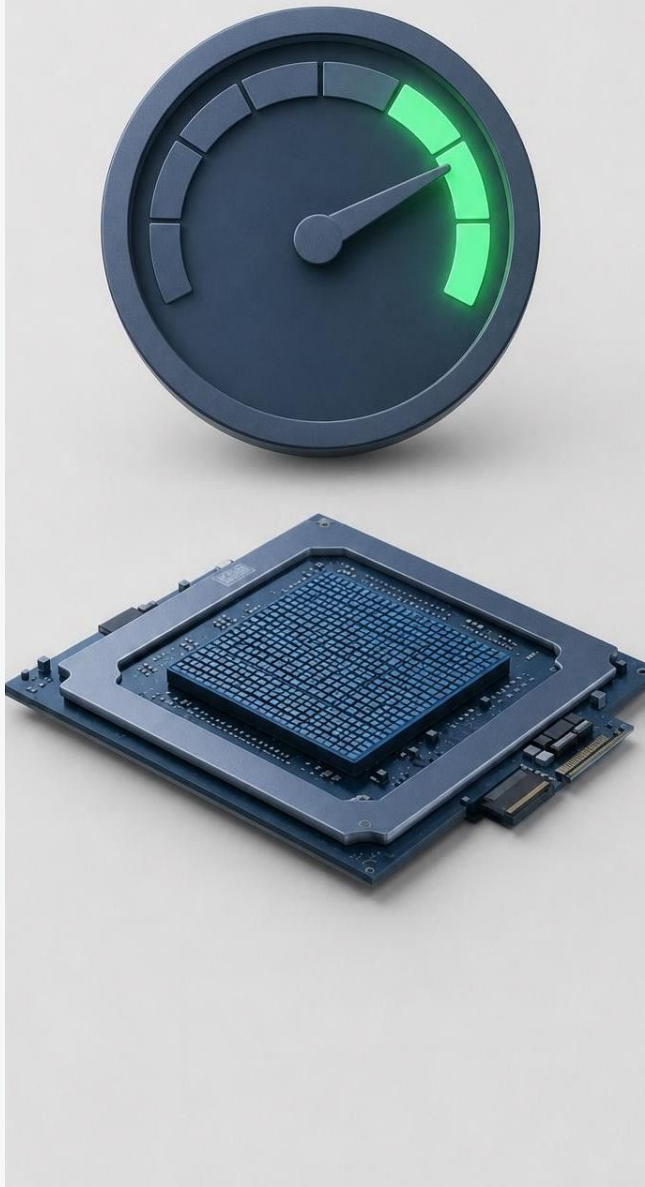
Case study 4: Claude as a kernel engineer

- Since early 2024, Anthropic's performance team hired with a take-home test: speed up a slow program on a **simulated accelerator chip**, like real kernel work (splitting work, reducing data movement, reorganising math). ~1,000 candidates took it; it hired most of the team.
- Team lead Tristan Hume found a pre-release **Claude Opus 4** beat almost all humans on the 4-hour version.
- The test was cut to **2 hours** with a faster start; **Claude Opus 4.5** matched the best human in that limit.
- Anthropic retired the test, built a new one, and published the original as an open challenge; with unlimited time the best human still beats Claude.

99×

From ~**147,734** cycles to **1,487** at launch.

Lessons: AI is a serious kernel helper for trying tile sizes and fusions; humans still set the ceiling for new ideas; benchmarks like KernelBench measure AI kernel writing.



How do we know if a kernel is good?

"2 milliseconds" means nothing without the best possible time, the **speed of light** of the hardware.

1. Memory-bound or compute-bound? Compare math steps per byte with the GPU's ratio (~295 on H100)
2. Best time: memory-bound = bytes moved ÷ bandwidth; compute-bound = math steps ÷ peak compute
3. Compare real time with best time

FlashMLA ~3,000 of ~3,350 GB/s ≈ 90%: excellent

FA-2 ~35% on H100, FA-3 ~75% · tuned matrix multiply **80–90%**

At 90%, tuning won't help much: move fewer bytes, use smaller formats, or change the algorithm. At 30%, lots of speed is waiting.

Kernel type	What we measure	Very good
Memory-bound	Bandwidth used ÷ max bandwidth	~80–90% or more
Compute-bound	Math speed ÷ peak math speed	~70–90% or more

Should you write your own kernel?

For most AI engineers, **usually not**, but know when that changes.

1

Use existing libraries and tools

PyTorch, cuBLAS, FlashAttention, vLLM, TensorRT-LLM: most projects stop here

2

Turn on a compiler

torch.compile: automatic fusion and Triton kernels with little effort

3

Write a custom Triton kernel

When one step is clearly slow and no library covers it

4

Write custom CUDA or CUTLASS

When every percent matters at very large scale

Worth it when

One operation takes a big share of time, it's far below its speed-of-light limit, your model has an unusual design (like DeepSeek's MLA), or scale makes a few percent real money

Probably not when

You haven't measured, a newer library may already solve it, or you run few GPUs

Rule: measure first. The kernel that feels slow isn't always the one that is.

Cheat sheet: the whole deck on one slide

- **Kernel:** a small program on thousands of GPU cores at once, each on its own data; a model is a long chain of kernels
- **Memory ladder:** registers and shared memory tiny and very fast, VRAM huge and slower; keep data near the top
- **Golden rule:** bring data from VRAM as few times as possible, use it as many times as possible while close
- **Who writes kernels:** NVIDIA (CUDA, cuBLAS, cuDNN, CUTLASS, TensorRT-LLM, FlashInfer) · Triton (OpenAI, block-based) · compilers (torch.compile writes Triton; XLA makes TPU kernels) · labs (DeepSeek for its own design; Anthropic across NVIDIA, TPU, Trainium)
- **Judge a kernel:** compare real speed with the speed of light for its limit

Trick	What it does
Fusion	Several steps in one kernel, no VRAM round trips
CUDA Graphs	Removes delay of launching kernels one by one
Tiling	Loads a tile once into shared memory, reuses it many times
Tensor cores	Uses special matrix units and small formats
FlashAttention	Attention without storing the giant score table

Remember: same GPU, model and math can run several times faster or slower depending only on the kernel. **And a fast kernel is only good if it gives the right answer.**